Briefings in Bioinformatics, 2026, 27, bbag440
https://doi.org/10.1093/bib/bbag440
Published: 22 August 2026
Case Study

OXFORD

De Bruijn graphs for pangenomics: in-depth performance benchmarking of de Bruijn graph-based tools for read mapping

Zülal Bingöl [1], Berkan Şahin [1], Klea Zambaku [1], Ricardo Roman-Brenes [1], Konstantina Koliogeorgi [2], Can Firtina [3], Onur Mutlu [2], Can Alkan [1,*]

[1]Department of Computer Engineering, Bilkent University, Bilkent, Ankara 06800, Türkiye
[2]Department of Information Technology and Electrical Engineering (D-ITET), ETH Zürich, Universitatstrasse 6, CH-8092 Zurich, Switzerland
[3]Department of Computer Science, University of Maryland, College Park, MD 20742, United States

*Corresponding author. Department of Computer Engineering, Bilkent University, Bilkent, Ankara 06800, Türkiye. E-mail: calkan@cs.bilkent.edu.tr

Abstract

De Bruijn graphs are widely used in pangenome representation due to their numerous advantages and extensions, such as colored and compacted variants that enhance the representation of genetic variation. Although de Bruijn graphs are becoming increasingly adopted, their performance and energy impact have not been clearly studied. Such an overlooked understanding can lead to suboptimal designs for de Bruijn graph-based tools in addressing the computational challenges posed by pangenome data. To identify workflow bottlenecks and assess the efficiency of hardware utilization, we present an in-depth performance analysis of state-of-the-art de Bruijn graph-based read mapping tools on pangenomic datasets, focusing on scalability of execution time, hardware resource utilization, and energy consumption. We observe that the tools primarily prioritize data parallelism for processing read datasets, disregarding the increasing complexity of the pangenome graph, which hinders scalability. As the pangenome graph grows in size and complexity, cache miss rates also increase, leading to poor overall performance. By extensively analyzing sources of suboptimal performance, we pave the way for optimizing the existing and future tools to fully realize their potential in advancing pangenome research.

Keywords pangenomics, de Bruijn graphs, benchmarking

Introduction

As high-throughput sequencing technologies continue to improve in accuracy, throughput, and cost efficiency, conventional treatment methods for diseases are being replaced by personalized solutions tailored to each patient. *Personalized medicine* strives to formulate customized treatments by inspecting the genetic background of individuals under a magnifying glass [1]. In this direction, the rapid development of sequencing technologies enables a growing number of organisms and species to be studied, paving the way for genomics to sample and analyze an increasing amount of data.

Genomics analyses typically compare a sequenced sample with the reference genome of the same species, using it as a template. However, a single reference genome does not adequately represent the species' diversity [2–6]. This lack of diversity introduces *reference bias* [5, 6], as it omits non-reference mutations [5] in various individuals, causing reduced alignment accuracy and eventually hindering downstream analyses such as personalized medicine. For example, the human reference genome GRCh38 lacks almost 10% of the genetic variations observed in humans [2]. This issue becomes even more pronounced when studying the genomic regions of high variations (e.g. African-ancestry alleles [7]) or the species with high genetic diversity (e.g. bacteria). To mitigate the reference bias created by employing a single reference genome, *pangenomes* [8] are introduced to better capture the diversity within a species or a population. A pangenome is a collection of sequences or genes of a particular species [2, 4, 8]. Using pangenomes instead of a single reference improves the downstream biological interpretation [9] by incorporating more alternatives to genomic regions and facilitating a better representation of structural variations [10].

Building on these observations, the recent advancements in human genome studies have reinforced the importance of human pangenome efforts. Long-read technologies have facilitated structural variation research by better encapsulating the repeat regions that were difficult to identify by short reads [11]. One of the most important recent achievements to better analyze all forms of genetic variation is completing the human genome assembly [12] by the Telomere-to-Telomere (T2T) consortium. Furthermore, the Human Pangenome Reference Consortium released the first draft for the

Received: January 07, 2026. **Revised:** July 19, 2026. **Accepted:** July 23, 2026
© The Author(s) 2026. Published by Oxford University Press.
This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial License (https://creativecommons.org/licenses/by-nc/4.0/), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited. For commercial re-use, please contact reprints@oup.com for reprints and translation rights for reprints. All other permissions can be obtained through our RightsLink service via the Permissions link on the article page on our site—for further information please contact journals.permissions@oup.com.

human pangenome reference [13] composed of 47 genetically diverse individuals. Since the efforts to collect more data to improve the diversity representation on pangenomes are building up rapidly [14–17], there is an urgent need to enhance the computational methods by understanding their performance bottlenecks to optimize them for handling future pangenome data loads.

Pangenome representations

Certain pangenome data structures are utilized to efficiently store and use large volumes of diverse genomic data. These data structures aim to (i) represent genomic variations for biological interpretability and (ii) enable fast search.

The simplest approach to representing a pangenome is to create a collection of linear genomes from different individuals. Such a dataset of linear samples may help to include the necessary information to exemplify each diverse element [8]. However, linear pangenomes have several critical disadvantages that make them an unpopular structural choice in pangenome studies. Firstly, linear pangenomes can create massive memory requirements in large genomes [18]. Since some parts of the genomes are also shared across individuals, linear genome collections store redundant data. Secondly, memory requirements limit the number of genomes stored in the pangenome. Thus, linear pangenomes may not contain enough variations for efficient diversity representation [19]. Last, these collections require extensive indexing techniques to help with computationally expensive data searches due to a high level of data redundancy.

Graph-based representations enhance storage efficiency and query performance while preserving genomic diversity in pangenomes. By compressing shared features and highlighting variations, they reduce storage requirements [9] and accurately represent alternative haplotypes, which are often misrepresented by linear genome collections [20]. Unlike linear representations, graphs effectively model complex variations, such as nested variants [2]. Consequently, graph representations are becoming more commonly adopted in pangenomes [20].

Among graph-based data structures, *variation graphs* and *de Bruijn graphs* are the most capable graph data structures to handle large amounts of data [9]. Variation graphs usually start with a reference genome as a backbone [21], and as more genomes are added, new variations are introduced to the graph with new node branches. The representation of pangenome graphs as variation graphs has been adopted in several recent studies, such as vg [22]. To identify the performance bottlenecks, Kaplan *et al.* [23] examine the performance of variation graph-based workflows.

de Bruijn graph-based pangenomes

As an alternative to variation graphs, de Bruijn graphs are gaining prominence in pangenome representation as a means to address some limitations associated with variation graphs [21, 24–27]. Unlike variation graphs, de Bruijn graphs mitigate reference bias by eliminating the need to designate a reference genome as a structural backbone during graph construction [21]. Constructed from k-mers, de Bruijn graphs offer greater stability than variation graphs, and their reproducibility enhances downstream analyses by producing more consistent outputs. Furthermore, a recent study highlights that de Bruijn graphs demonstrate superior scalability with increasing input sizes [9]. As de Bruijn graphs emerge as a powerful foundation for the future of read mapping in pangenome studies, our investigation centers on de Bruijn graph-based tools.

Briefly, in a typical workflow, the de Bruijn graphs are constructed using a set of genomes or alternative haplotype sequences [24, 25]. The resulting graph is then indexed to enable efficient queries [25, 28]. Mapping reads to the de Bruijn graphs then follows the seed-and-extend strategy, which is also common in linear genome mapping since the introduction of BLAST [29]. Different sequence-to-graph (S2G) aligners employ various design choices, including their own graph indexing methods. However, all S2G aligners first find an exact match of a short substring in the query read to an edge in the graph (seed) and then try to extend the alignment using a dynamic programming (DP) algorithm, such as variants of partial order alignment [30]. Examples of the state-of-the-art de Bruijn graph construction and indexing tools include Bifrost [25], TwoPaCo [24], Pufferfish [28], and MetaGraph [31]. One of the most popular general purpose S2G aligners is GraphAligner [27], where PuffAligner [26] depends on Pufferfish for indexing the graph, and MetaGraph is an all-inclusive de Bruijn graph construction, indexing, and alignment suite. For simplicity, we will refer to the Pufferfish [28]+PuffAligner [26] tool pair as Pufferfish for the rest of the paper.

Despite the advantages, graph-based pangenomes introduce some challenges that should be addressed to highlight their future potential. As the number of genomes increases, graph size and complexity expand exponentially, necessitating refinement techniques and tailored algorithm designs for read mapping to overcome the challenges. In this paper, we present a detailed performance analysis of the state-of-the-art de Bruijn graph-based read mapping tools, GraphAligner [27], MetaGraph [31], and Pufferfish [26, 28], to demonstrate the impact of performance bottlenecks in the read mapping pipeline targeting pangenome references. We therefore aim to provide a guideline for further improvement of de Bruijn graph-based read mapping to achieve faster and more energy-efficient pangenome analyses.

Briefly, this paper makes the following contributions:

- We provide the performance insights from three state-of-the-art de Bruijn graph based tools (GraphAligner [27], MetaGraph [31], and Pufferfish [26, 28]) that perform S2G mapping.
- We examine the scalability of tools from two angles: (i) in terms of the number of threads utilized and (ii) the size and complexity of the pangenome graph.
- By programmatically isolating the three common stages (i.e. *indexing*, *seeding*, *alignment*) of S2G mapping, we collect fine-grained hardware performance counters to show how the tools' methods utilize resources and how these values are reflected in the overall performance.
- We analyze how the number of threads and pangenome size correlate with the energy consumption of isolated stages.
- The graph structure is another important factor impacting the mapping performance. We conduct experiments in two different species whose pangenome reference graphs exhibit structural differences, as they contain varying levels of variation: *Escherichia coli* graphs with more tangled structure, and *Homo sapiens* Chromosome 22 (i.e. Human-Chr22) graphs with fewer nodes, mostly in a linear layout.

Evaluation methodology

To pinpoint the bottlenecks per stage, we decompose workflows into three common stages: *indexing*, *seeding*, and *alignment*. Although indexing is usually performed once, it is critical and therefore included

in our analyses. Tools were run with their default parameters (Section S4). For MetaGraph and Pufferfish, which construct the graph within their pipelines, measurements begin at the start of their indexing stage; Pufferfish was evaluated in its default dense index mode for faster k-mer lookups. For GraphAligner, we treat the construction of its intermediate alignment graph, which inserts breakpoints to facilitate traversal, as the indexing stage.

Rapid growth in pangenome data demands scalable tools capable of processing substantial loads effectively. Therefore, we assess the tools' scalability from two angles. Firstly, we gather metrics from pangenome graphs of varying sizes to observe their scalability based on the graph's data size and complexity. Secondly, we conduct tests with different thread counts to evaluate the efficiency of parallelizing the workflows. We provide observations on both the isolated stages (i.e. indexing, seeding, and alignment) and end-to-end workflows. For isolating the stages, we follow the respective tool's own flags (e.g. start of seeding) for designating the code blocks. Through this approach, instead of focusing on a single kernel function per stage, we provide analyses encapsulating all necessary procedures.

We focus our experiments on two organisms: *E. coli* and Human-Chr22 (see Section S1 for dataset details). Due to the significant diversity in bacterial genomes, the pangenome reference graphs for *E. coli* are more intricate, featuring shorter segments and numerous edges. Conversely, the pangenome graphs for Human-Chr22 have extensive sequential sections and a simpler structure (see Section S1.2 for graph statistics). This difference in graph structure between the pangenome reference graphs enables us to study graphs of varying complexities and identify specific areas where the performance may be weaker. Please refer to Section S9 for other human graph experiments.

NUMA-aware testing environment

Most modern computing systems employ Non-Uniform Memory Access (NUMA), where multiple processors on a single motherboard have access to both local and remote memory nodes. NUMA-aware architectures can improve overall performance; however, the latency of remote node access is higher than that of local node access [32, 33]. In addition, the operating system scheduler can assign a thread to different processors during its execution lifetime. Together with NUMA design, this can lead to misleading performance counts in the benchmarking. To gain deeper insights into the performance, we provide a NUMA-aware testing environment using `numactl` [34] and set the CPU thread affinity to ensure that the observed processes run on the designated processors and use their local memory nodes.

Hardware performance counts

We collect performance metrics using both a profiler as a wrapper (i.e. Linux `perf` [35]) and adding code routines (i.e. `perf event attributes` [36]) to extract specific performance counters within the areas of interest. To obtain hardware performance counts of the isolated stages, we inject `perf event` [36] routines into the tools' codebase and collect the values of cache references, cache misses, branch instructions, and branch mispredictions, as these counts can be traced per process. Since these code routines add overhead to the overall performance, we measure hardware performance counters alone; then test time and energy separately without the `perf` code routines. For full workflow metrics, such as instruction per cycle (IPC) and peak memory, we use the commandline-based `perf` [35] and `/usr/bin/time` wrappers.

Function-specific hardware performance counts

We conduct further in-depth cache analysis using the DAMOV-SIM simulator [37], which combines CPU and RAM simulators [38, 39], on the target functions we identified with `perf`. We collect cache misses, hits, and miss ratios across three cache levels and IPC. Augmenting the results obtained via `perf` with DAMOV enables the analysis of cache usage patterns during the execution of a specific function, due to start and stop markers in code. The data obtained from DAMOV are also free from the influence of other processes running during benchmarking because the entire cache hierarchy is simulated. The details of the system simulated by DAMOV are given in Table S11.

Energy consumption

Performance profiling tools, such as `perf`, can provide the energy consumption measurements for the entire execution. To obtain the fine-grained energy values of the stages, we employ an alternative approach by utilizing Intel's Running Average Power Limit (RAPL) [40]. RAPL is a hardware feature that provides insights into the real-time power consumption of the CPU package and DRAM during execution [41, 42]. By using `msr-tools [43]` with RAPL, we read model-specific registers at the start and end of each stage to estimate the energy expenditure of the isolated code portions. These registers hold the values for the entire socket, not per process. Therefore, we report seeding and alignment together as a sum, since usually individual threads execute these stages consecutively. Additionally, the NUMA-aware testing mechanism prevents the addition of unused socket values to the total, providing a more realistic measurement.

Graph construction

To identify areas of potential improvement in the S2G mapping pipelines, we conduct a series of experiments by constructing single-reference graphs and pangenome graphs from the assemblies described in Section S1. Throughout the paper, graph complexity refers to the number of assemblies used to construct the graph, such that a graph built from more assemblies is considered more complex than its comparative counterpart (e.g. 500-assembly versus 100-assembly graphs). As the underlying graph has an impact on the overall performance, we primarily report the results with the graphs on which the tools show their best performance. To broaden the scope, we also provide our observations on the same graph in Section S8.

MetaGraph [44] constructs SuccinctDBG by default and indexes it. Pufferfish [28] internally uses TwoPaco [24] to build a compacted de Bruijn graph and creates an index. Since GraphAligner does not build a graph, we generate compacted de Bruijn graphs using Bifrost [25], which produces versatile de Bruijn graphs in the widely adopted GFA format [45].

Results and discussion

Overall workflow

We begin with a high-level assessment, measuring end-to-end performance via wall clock time and peak memory usage. For MetaGraph and Pufferfish, we extract the indexing time from graph construction and indexing runs, and add it to the end-to-end runtime of mapping.

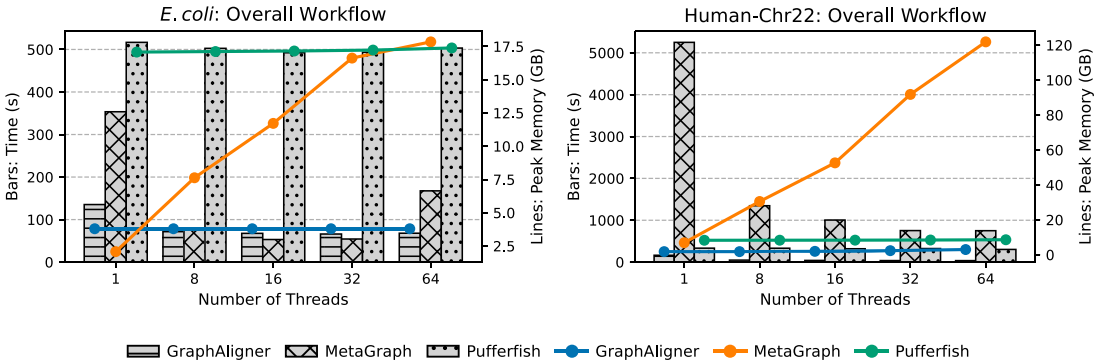


Figure 1 End-to-end time and peak memory on the largest pangenome graphs with PacBio reads.

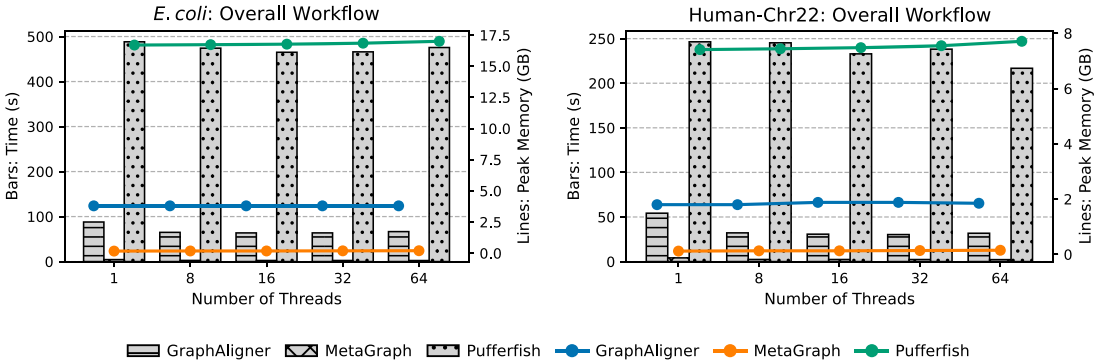


Figure 2 End-to-end time and peak memory on the largest pangenome graphs with Illumina reads.

Similarly, we consider the peak memory of their mapping stage to be the peak memory, since more RAM is allocated to graph construction than to indexing. For simplicity, we include PB (PacBio HiFi) reads results in the main part, but refer to the Illumina and ONT (Oxford Nanopore) diagrams provided in the Supplementary.

We also provide brief interpretations of the performance in relation to mapping rates of the tools, as the percentage of reads with a mapping over all 1000 reads in the datasets (full tables can be found in Section S3). Since the detailed evaluation of the tools' mapping accuracies is out of the scope of this paper, we refer to Coggi *et al.* [46] for the comprehensive accuracy evaluation.

Scalability in multithreading

Figures 1 and S1 demonstrate that GraphAligner takes the shortest time with the lowest memory usage of under 4GB, where MetaGraph and Pufferfish show different trends for *E.coli* and Human-Chr22 graphs in long reads. Being primarily a metagenomics tool, MetaGraph runs on the tangled *E.coli* graphs much faster than on mostly linear Human-Chr22 graphs. On the contrary, Pufferfish's performance on Human-Chr22 graphs is better both in terms of time and peak memory. For short reads, MetaGraph runs fastest with the lowest peak memory under 1GB (Fig. 2). GraphAligner shows less scalability in threading compared with long reads, whereas Pufferfish has similar runtimes for *E.coli* since indexing majorly dominates the overall performance (see "Execution Time Breakdown").

As the number of threads increases, Pufferfish does not scale well, finishing up in almost equal overall time. GraphAligner and MetaGraph exhibit a better parallelism with a decreasing trend. Among all the tools, MetaGraph has the highest memory usage for long reads, reaching over 768GB with Human-Chr22 ONT reads.

Table 1 Mapping rates in *E. coli* and Human-Chr22.

	E. coli				**Human-Chr22**			
Reads	**Size**	**GA**	**MG**	**PF**	**Size**	**GA**	**MG**	**PF**
ILMN	1	82%	100%	76%	1	34%	100%	96%
	1485	99%	100%	77%	84	100%	100%	96%
PB HiFi	1	92%	100%	53%	1	100%	100%	50%
	1485	100%	100%	94%	84	100%	100%	73%
ONT	1	84%	100%	5%	1	96%	NA∗	17%
	1485	99%	100%	19%	84	100%	NA∗	26%

Size refers to the number of assemblies in pangenome graph GA, GraphAligner; MG, MetaGraph; PF: Pufferfish. ∗We could not finish MetaGraph runs due to high RAM usage (> 768GB)

Scalability in pangenome size

Increasing the number of assemblies can significantly improve mapping results by creating more nodes and connections in the graph (Table 1). On the other hand, the cycles emerging from these structures increase the graph's complexity for traversals. As Figs 3, 4 and S2 illustrate, higher graph complexity puts more stress on the overall performance of GraphAligner and Pufferfish. MetaGraph exhibits negligible slowdown for both species, processing the short reads the fastest. However, it is the slowest with long reads, especially in Human-Chr22 graphs, with the highest memory usage.

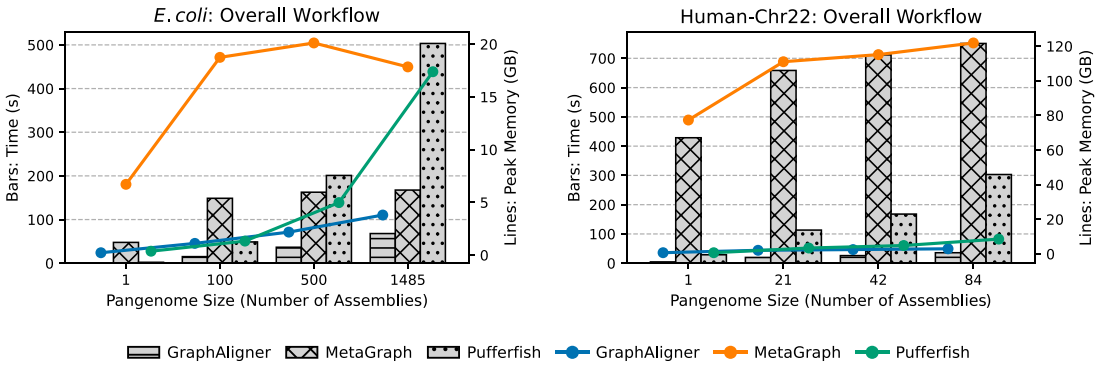


Figure 3 End-to-end time and peak memory with 64 threads, PacBio reads.

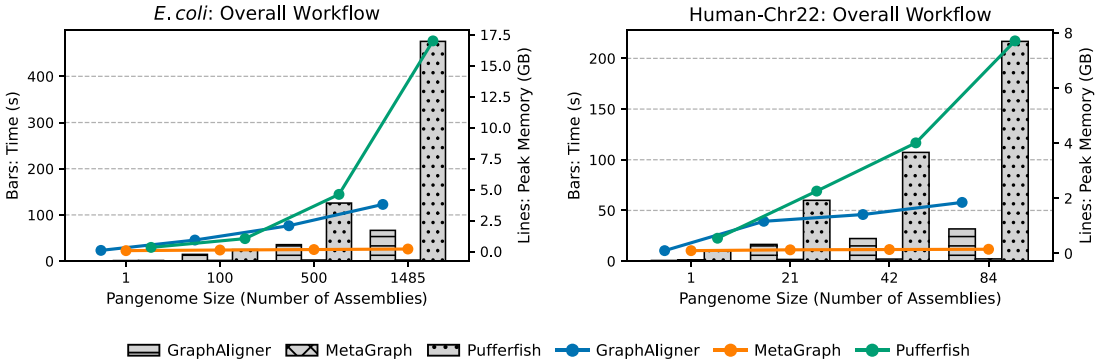


Figure 4 End-to-end time and peak memory with 64-threads, Illumina reads.

Execution time breakdown

By breaking down the overall single-thread execution time (Fig. 5), we find that the longest duration for each tool occurs in different stages. Alignment massively dominates MetaGraph by 98.4% and has 54.9% share of GraphAligner's runtime. Pufferfish's indexing is the most time-consuming part of the workflow, with 67.5%, while alignment takes only 17.5%.

Indexing

Indexing is indispensable for efficient read mapping, despite being a one-time procedure for most software. How graph data are indexed has a great impact on the seeding and alignment performance. Furthermore, indexing itself can be a time-consuming and exhaustive process.

Figure 6 reflects that Pufferfish takes the longest time in indexing and cannot scale up linearly with the increasing number of threads. Upon retrieving the graph from TwoPaco [24], it stores the unipaths (i.e. non-branching paths) in a table and in a packed 2-bit encoded array, maintaining the addressing relationship between the table and the array through ranking in a bitvector. By building a minimal perfect hash function (i.e. MPHF) with BBHash [47], it creates an index of all possible kmers on the encoded array of unipaths. During these steps, no parallelism is applied, so indexing always runs on a single thread.

After receiving a bidirected graph as input, GraphAligner converts it into intermediate directed acyclic graphs for forward and reverse directions. Unlike Pufferfish's unipath-based approach, it constructs a minimizer index where each minimizer stores its corresponding node information. Designating the kmer ranks in bitvectors for minimizers and dividing minimizers into buckets, it builds an MPHF per bucket using BBHash [47]. Although it processes buckets in a multithreaded fashion, the indexing step is mostly performed on a single thread due to the dominance of the single-threaded function for alignment graph construction, `sparsenComponentNumbers`, which is called exactly once before other threads are spawned. Therefore, indexing does not scale linearly with the increasing number of threads.

MetaGraph's indexing [44] begins with a single joint de Bruijn graph, represented as a BOSS table [48] of a kmer dictionary, and constructs an annotation matrix to encode the membership of kmers in different samples. During the annotation matrix construction, it processes multiple samples simultaneously through parallel iteration. Instead of traversing all kmers, it uses unitigs from the cleaned sample graphs, extracted during graph construction, to avoid iterating over identical kmers. This parallel, redundancy-avoidant strategy significantly reduces the time required to create the annotation matrix, making MetaGraph's indexing the fastest to complete and the most scalable among the tools.

Examining the cache utilization during indexing of the largest pangenome graphs (Fig. 6), the single-thread MetaGraph exhibits a notably lower cache miss rate of 3%, whereas GraphAligner and Pufferfish exceed 45%. Increasing the thread count predictably raises MetaGraph's miss rate due to high parallelism, but it still keeps it below 30% mostly. 64-threads uses two sockets, which contributes to the slightly higher rate. We observe that efficient concurrent processing, combined with harnessing data locality by operating on sorted kmers, gives MetaGraph's indexing performance a significant edge. Since GraphAligner and Pufferfish's indexing is not scaled with increasing thread counts, their miss rates do not change noticeably in a

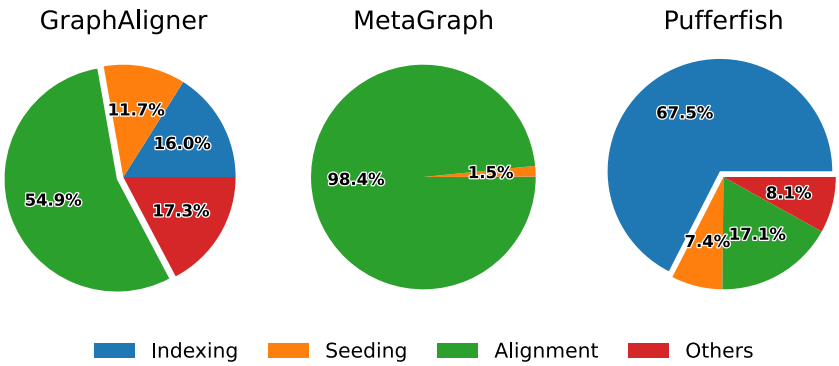


Figure 5 Human-Chr22: execution time breakdown with single thread in the largest pangenome graphs with PB reads.

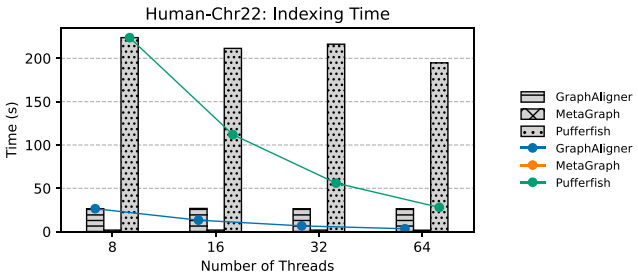


Figure 6 Indexing largest Human-Chr22 graphs. Lines indicate the expected time values on a linear scale.

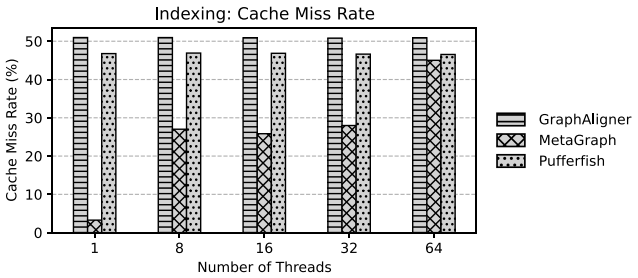


Figure 7 *E.coli*: cache miss rates in the largest pangenome graph.

multithreaded environment. High cache miss rates contribute to longer durations by requiring costly memory accesses.

Enriching the pangenomes by increasing the number of assemblies included is expected to result in longer indexing times and increased resource allocations. In terms of storing the index structure (Table S8), MetaGraph's graph and index occupy 31 MB for a single genome, and grow to 60 MB for an 84-assembly graph for Human-Chr22. In comparison, Pufferfish's dense index takes up 203 MB and 5.5 GB of space. GraphAligner uses an on-the-fly indexing approach and does not export the index structure. Pufferfish's unipath table stores detailed information on the non-branching path per assembly; therefore, traversing all paths in the graph and creating the kmer relations requires significantly longer durations when there are more assemblies in the graph (Fig. 8). MetaGraph, on the other hand, traverses on unitigs; thus, only kmer membership relations are updated on the annotation matrix if the newly added assemblies introduce different unitigs. Both on single-thread and multi-thread, growing the pangenome from a single assembly to 84 assemblies makes only a few seconds of difference. This is also supported by the low cache miss rates: while GraphAligner shows a gradual increase due to many random accesses during alignment graph conversion, MetaGraph's cache miss rates show a negligible rise in comparison (Fig. 8, right).

Seeding

Identifying seed hits requires numerous random accesses to the graph. Depending on the data structure and query memory access patterns, seeding performance can vary significantly. Seeding usually includes filtration and clustering designed to reduce the alignment costs; however, these additions expand the overall workload of the seeding stage.

Seeding PB reads on Human-Chr22 takes the shortest time for GraphAligner (Fig. 9). It starts by finding the kmer's index bucket and querying the kmer's rank in the bucket's MPHF, which leads to the kmer's instance in the bucket's sorted array. All possible kmer positions are collected by iterating over the sorted array, and the least frequent ones are chosen as seeds. Using these seed hits, GraphAligner detects the superbubbles on the directed acyclic graph, creates chains of superbubbles by linearizing the denoted subgraph nodes, and then clusters seeds and orders the clusters based on the scores. Each thread runs these operations for each read; therefore, multithreading directly increases the simultaneous operations.

Since Pufferfish has unipath-based indexing, it starts by querying kmers from the read in the unipath position vector using MPHF. Once exactly matching unipaths are located, the kmer is then extended to a uni-MEM (unitig maximal exact match [49]) with linear scan. After the uni-MEMs are collected, highly repetitive ones are filtered out. Figure 9 shows that seeding with Pufferfish does not scale up with multithreading and takes the longest time to complete.

Indexing the kmer ranges in the BOSS table by suffixes improves MetaGraph's kmer query performance. Once the kmers are found, MetaGraph also adopts a uni-MEM based approach and creates the seeds by combining the consecutive kmers on the unitigs. With batched kmer queries on the annotation matrix, MetaGraph's seeding duration scales better with concurrency, leading to a shorter time than Pufferfish.

Pufferfish's cache miss rates are lower than the other tools on average (Fig. 10). Storing all hit positions of a k-mer within a single row of the unitig table may enhance temporal locality during querying. GraphAligner's seeding mechanism runs concurrently per read, providing rapid access to the desired index bucket. While this containerized approach facilitates concurrency, it also lowers data reuse per thread. Additionally, when the graph nodes are longer, like in Human-Chr22 compared with *E. coli*, traversing the nodes to find the superbubbles or forming the seed clusters requires

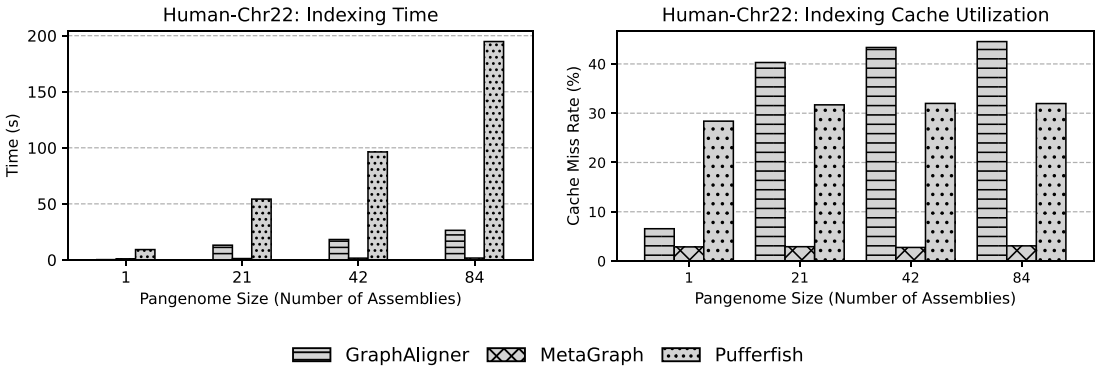


Figure 8 Left: indexing duration with 64-threads, Right: cache miss rates of a single thread for indexing.

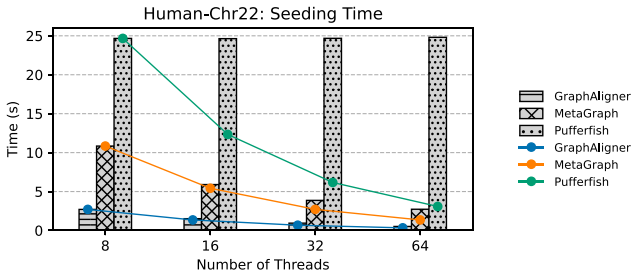


Figure 9 Seeding time on the largest pangenome graphs with PB reads. The lines indicate the expected time values on a linear scale.

random accesses to memory, causing more frequent cache misses (Fig. 10).

By querying kmers in the annotation matrix in a batched fashion, MetaGraph aims to improve cache locality. This strategy is likely to support Human-Chr22 (Fig. 10, right), as unitigs are longer, allowing consecutive kmers to fit. However, in *E. coli* graphs, it is harder to fit numerous kmers coming from long PB reads into shorter unipaths, which leads to higher cache miss rates by decreasing the reuse of unitigs.

Introducing more assemblies to the graph with more variation creates branches and more alternative paths with shorter node segment lengths (Tables S3 and S5). On both *E.coli* and Human-Chr22 with a single thread, we observe that GraphAligner's cache miss rate is usually the highest and increases gradually as more assemblies are introduced to the graph (Figs 11 and 12), even though it consistently takes the shortest for seeding. To investigate the hardware events further, we identified that the `initMinimizers` function's duration is the bottleneck of seeding by using `perf`. Monitoring this function with DAMOV without memory prefetching reveals that L1 cache miss rates are low across all thread counts ($\sim$ 5%). However, L2 cache miss rates are as high as 91.7% for a single thread, decreasing as the thread count increases, and L3 (i.e. LLC) cache miss rates are lower than L2 (54% for a single thread). Since both L1 and L2 caches are private to the cores, cores that execute code with less random memory patterns incur fewer misses, lowering the average L2 miss rate. In addition, L2 MPKI is low for both single-thread and 64-thread runs, at 1.74 and 1.87, respectively. The low MPKI values suggest that there are few L2 accesses, but most of those accesses miss L2, probably because L1 misses go straight to the L3 or memory. Streaming data processing can be one of the culprits behind poor L2 cache reuse in GraphAligner. With memory prefetching, simulated cache miss rates become 4.0% for L1, 73.0% for L2, and 55.3% for L3. These results show that memory prefetching helps recover the L2 cache miss rate to 73.0%, possibly facilitating faster execution by reducing seeding's high cache miss rates. GraphAligner's minimizer-based indexing makes kmer lookup depend more on the number of possible positions than on node length since it adds a breakpoint every 64 base pairs, so clustering seeds across many nodes increases random accesses and raises L2 miss rates. Despite these high L2 misses, the cache still recovers many accesses because GraphAligner processes seeds in a streaming, data-parallel manner, which yields the shortest overall processing time. MetaGraph and Pufferfish show steadier cache miss growth and better scalability on Human-Chr22, as seeding on longer unipaths helps balance miss rates. In *E. coli* graphs, MetaGraph has higher miss rates, likely due to reduced reuse of shorter unitigs.

Alignment

Alignment is the most computationally expensive stage because it relies on DP algorithms. Graphs' nonlinear structure forces DP to consider all paths at branches, and cycles or nodes with many outgoing edges further complicate S2G alignment, so sequence-to-sequence methods cannot be applied directly. Tools therefore either implement graph-native alignment techniques or use heuristics to avoid alignment when possible to improve performance.

In GraphAligner, a read's thread continues with alignment right after receiving the seed clusters. GraphAligner adopts a variation of a DP algorithm [50] that combines Myers' bit-parallel approach [51] with Navarro's S2G alignment [52] for extending the seeds on the graph. Using a novel banding scoring technique, it calculates the DP matrix by traversing each node's branch in slices of 64 rows. Even though operating on branching nodes with DP matrix induces random accesses, this sliced approach appears to help as GraphAligner demonstrates the lowest cache miss rates in both Human-Chr22 and *E.coli* with PB reads (Figs 15 and 16). It stores the node IDs and the border cells of each alignment slice of the DP matrix in a hash table. In this way, GraphAligner has read-centric data parallelism; therefore, increasing the number of threads can reduce the time required to align PB reads (Fig. 13). We observe that in Human-Chr22, GraphAligner's alignment duration scales better with the number of threads, providing a 6*x* speedup from 8 to 64 threads. In contrast, MetaGraph is 4*x* faster, and Pufferfish finishes around the same time.

We analyze MetaGraph's Single-label Seed-Chain-Extend Alignment (i.e. SCA) [31], which is the default method, and do not compare the tools for multilabel alignment. MetaGraph starts with anchoring the

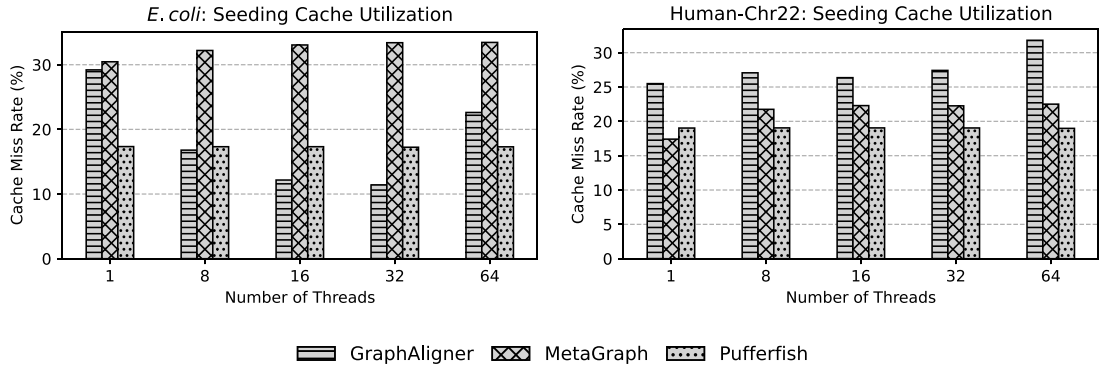


Figure 10 Seeding cache miss rates on the largest pangenome graphs with PacBio reads.

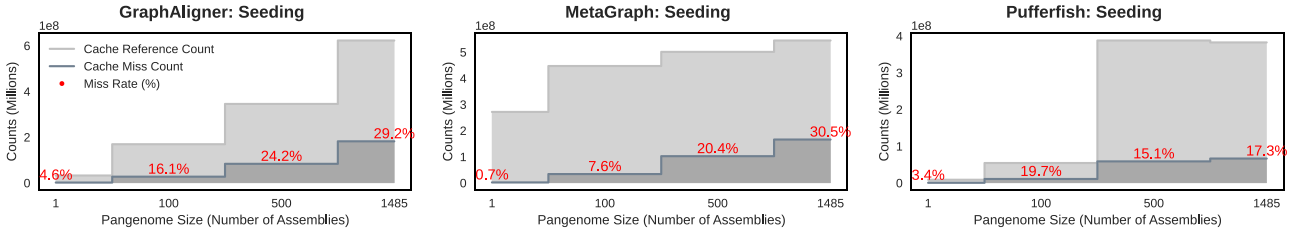


Figure 11 *E.coli*: cache utilization in seeding with single thread and PacBio reads.

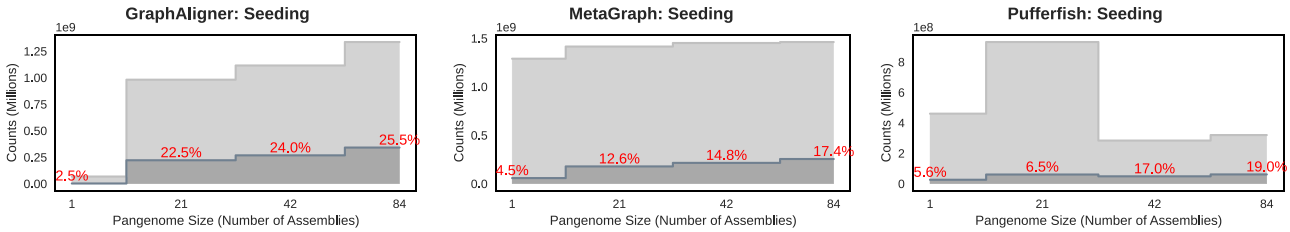


Figure 12 Human-Chr22: cache utilization in seeding with single thread and PacBio reads.

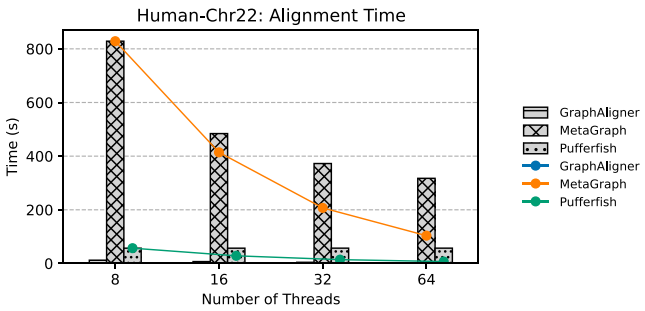


Figure 13 Alignment time of PacBio reads on the largest pangenome graph. The lines indicate the expected time values on a linear scale.

seeds and sorting them. After merging the anchoring into maximal unique matches, it builds chains from anchor groups. Between consecutive anchors, it extends the anchors via global alignment with TCG-Aligner [53] and eliminates overlapping chains with completed alignments. MetaGraph divides reads into batches for threads, and each thread runs alignment after seeding per read in its batch. Like GraphAligner, it adopts data parallelism for read processing. Higher cache miss rates can contribute to the lower alignment scalability of PB reads.

Unlike GraphAligner and MetaGraph, Pufferfish's alignment executes on linearized sequences rather than directly on the graph nodes. It constructs co-linear chains from filtered uni-MEMs using DP. The high-scoring chains of linearized paths on the graph are selected for alignment. Using the long matches of MEM chains between the reference and the read, it first determines the optimal alignment via global alignment with KSW2 [54] within the intervals, and stitches these regions together with exact matches at the borders. To speed up the alignment, several heuristics are applied, including skipping the alignment if a region is already detected as previously aligned for the same read, early stopping the alignment if a valid score cannot be achieved, and employing banded alignment. These alignment elimination techniques give Pufferfish an advantage over MetaGraph in terms of alignment time, but the early stops may contribute to the lower mapping rates (Table 1).

Figures 15 and 16 demonstrate the cache miss rates of the tools during aligning PB reads. As the assembly content of the pangenomes increases, GraphAligner's IPC (single thread) drops from 2.47 to 1.82 and from 2.36 to 1.98, respectively, indicating an increase in compute intensity. However, it exhibits the most scalable control over cache miss rates, which in turn reflects the duration. Pufferfish has 42.1% of miss rate in *E.coli* pangenome of size 500, but the miss rate decreases on the largest pangenome graph as it makes fewer alignment attempts. With a high IPC (> 3), Pufferfish becomes compute-bound and exhibits inefficient cache utilization. On the other hand, Metagraph exhibits lower IPCs, 1.73 and 1.27, for the largest *E. coli* and Human-Chr22

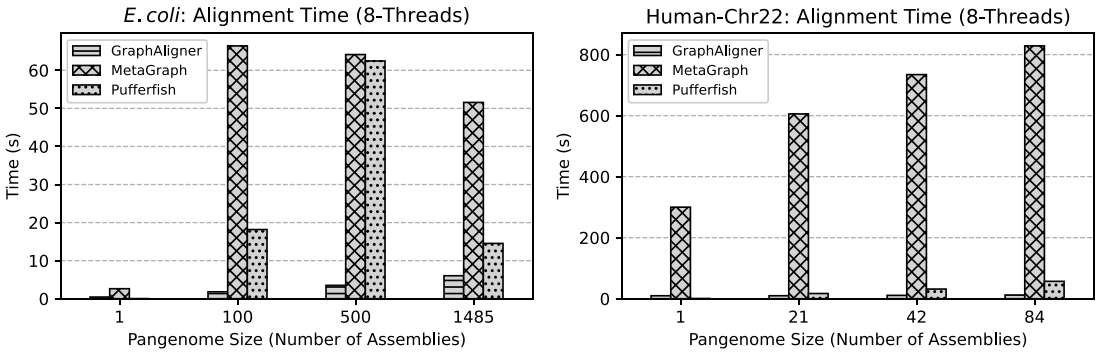


Figure 14 Alignment time of PacBio reads on different-sized graphs.

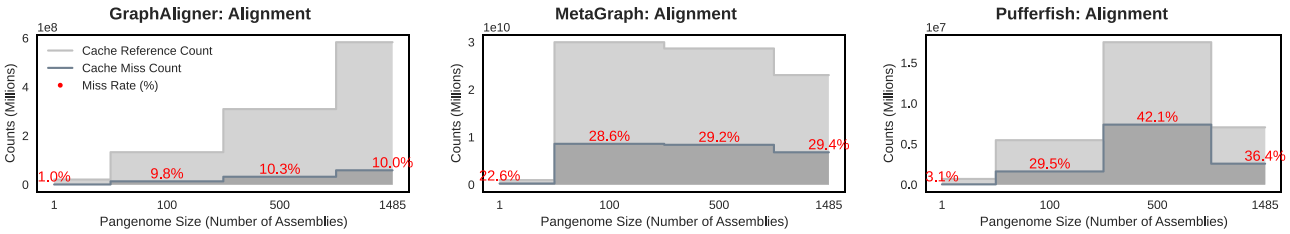


Figure 15 *E.coli*: cache utilization in alignment with single thread and PacBio reads.

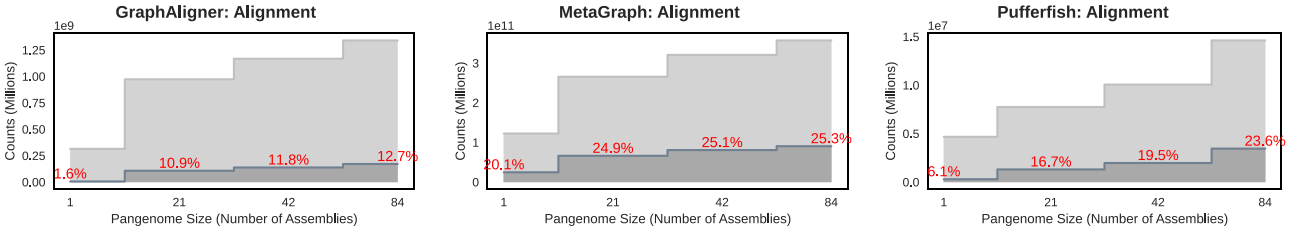


Figure 16 Human-Chr22: cache utilization in alignment with single thread and PacBio reads.

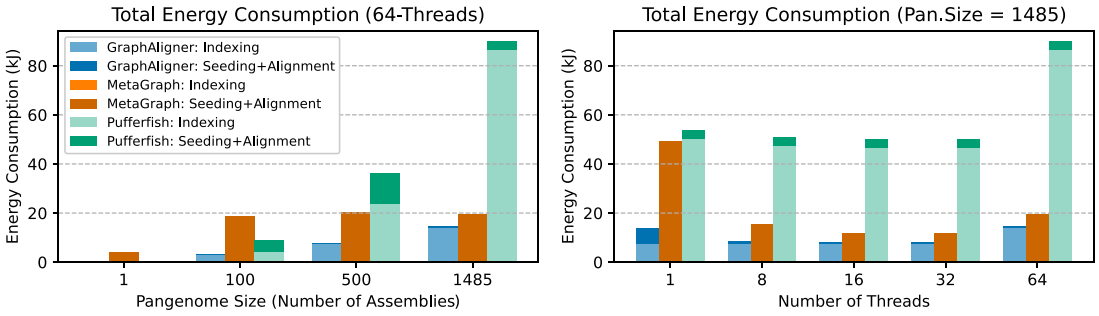


Figure 17 *E.coli*: total energy consumption with PacBio reads.

graphs, respectively, with high cache miss rates above 25%, which can signal the memory-intensity of the workload, and cause the longest alignment durations (Fig. 14).

Energy consumption

We evaluate energy consumption using the Intel RAPL library [55] by examining two components: the package energy (i.e. PKG), which encompasses the entire socket, including the last-level cache, cores, and memory controller; and DRAM energy, which is consumed by the random access memory.

Inspecting energy consumption across the indexing, seeding, and alignment stages, we observe that the *duration* of these stages is

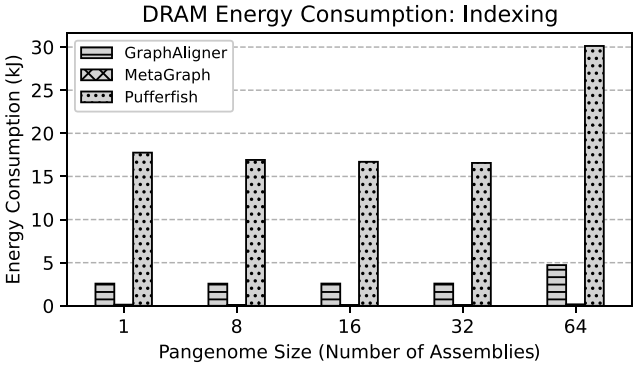


Figure 18 *E.coli*: DRAM energy consumption in indexing largest pangenome graph.

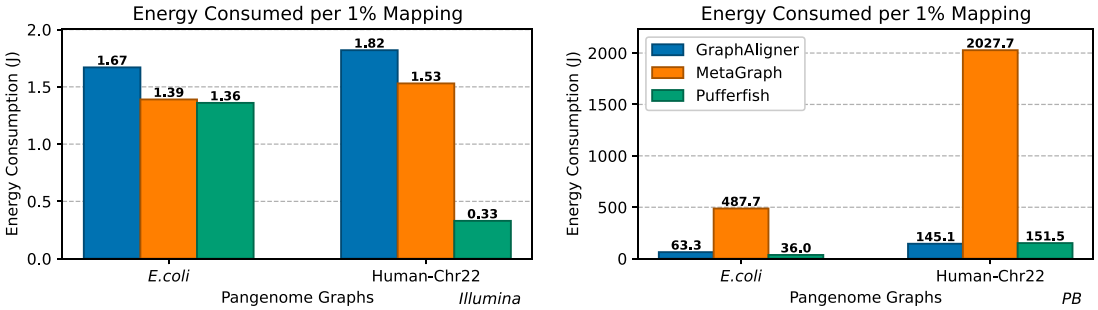


Figure 19 Single thread's energy consumption of seeding and alignment on the largest pangenome graphs, mapping 1% Illumina(left) and PacBio (right) reads.

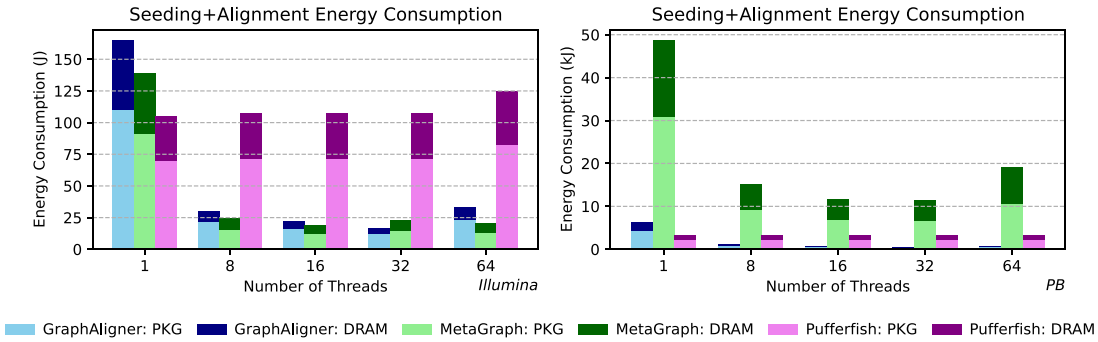


Figure 20 *E.coli*: seeding and alignment energy consumption on the largest pangenome graphs with Illumina(left) and PacBio (right) reads.

the most significant factor. The total energy consumed by the package and DRAM (Fig. 17) follows similar trends to the durations (see "Overall workflow") . Processing *E.coli* graphs with more threads, GraphAligner, and MetaGraph complete all stages in shorter times, thereby spending less energy compared with running on a single thread (Fig. 19). Because Pufferfish does not parallelize indexing efficiently, it consumes the largest total energy. Indexing accounts for the majority of total consumption for both GraphAligner and Pufferfish. For Human-Chr22 graphs, MetaGraph's seeding and alignment take a much longer time, causing more energy to be spent.

Figure 18 shows the energy values spent by the random access memory in indexing, which correlates with the cache miss rates in Fig. 7. As the cache miss rates increase, frequent and numerous accesses to DRAM for an extended duration make Pufferfish consume more energy. Even though GraphAligner's cache miss rates are high, DRAM still consumes less energy than Pufferfish due to its shorter time.

Pufferfish's single thread is the most energy efficient to achieve 1% of mapping when indexing is ignored (Fig. 19), due to the early stops in the alignment. MetaGraph and GraphAligner compensate their thread-level energy inefficiency by facilitating faster runs with parallelism.While seeding and aligning long reads to both *E.coli* and Human-Chr22 graphs, GraphAligner consumes the least amount of energy for seeding and alignment since it takes the shortest time (Fig. 20). MetaGraph spends the most time in alignment and consumes the highest energy. For short reads (Fig. 20), even though MetaGraph is fastest to execute, its energy consumption is on par with GraphAligner, likely due to the higher cache miss rates in seeding (Figure S4). In both cases, as the number of threads increases, energy consumption decreases, along with runtime, highlighting the importance of parallelism.

Conclusion

Pangenome references are quickly replacing single reference genomes due to their ability to reduce reference bias. De Bruijn graph-based tools play a critical role in pangenome analysis due to their practical capabilities, such as haplotype tracing on colored and compacted graphs. While de Bruijn graphs are increasingly being adopted, their performance and energy impact remain insufficiently studied.

In this work, we conduct an in-depth investigation of the performance of the state-of-the-art de Bruijn graph-based read mapping tools. We evaluate their scalability across different pangenome graph sizes and complexities and identify performance bottlenecks. MetaGraph is the fastest for short reads, exhibiting better multithread scalability and larger pangenomes for both *E. coli* and Human-Chr22 graphs. However, its alignment is memory-intensive with respect to read length, and this heavily impacts its overall performance as read length increases. Being primarily a taxonomic classifier, it consistently finds a mapping for each read, achieving a 100% mapping rate across all our tests. GraphAligner's mapping rate improves as pangenome graphs grow with more assemblies. Its long read processing is superior to other tools due to the efficient application of banded partial order alignment and fast bitwise DP operations. Except for indexing, it demonstrates structured concurrency with scalable performance for both seeding and alignment. On short reads, its performance is weaker than MetaGraph's, largely due to the time-consuming indexing procedure. GraphAligner's bucket minimizer-based approach helps achieve its lowest seeding duration. Pufferfish's performance is mostly hindered by inefficient concurrency, which is more pronounced in indexing than in other stages. Even though it exhibits lower seeding cache miss rates than GraphAligner, using a single-pointer

MPHF appears to create contention among queries, thereby increasing the time taken.

Domain-specific hardware architectures can further accelerate de Bruijn graph-based read mappers beyond software and hardware-friendly optimizations. When data reuse is low (e.g. indexing), graph-aware prefetchers can predict traversals and fetch kmer/unitig relationships to reduce cache misses; irregular query patterns during seeding worsen cache pressure, which hardware–software co-design can mitigate by restructuring memory access and using specialized hierarchies. Combining on-chip scratchpads with High-Bandwidth Memory (HBM) enables highly parallel, lower latency accesses (e.g. SeGraM [56]); HBM sustains alignment concurrency while bitwise DP accelerators (e.g. deBGA [49], SeGraM [56]) cut computational redundancy. Processing-in-Memory offers an alternative by performing computations inside memory arrays to eliminate costly DRAM-CPU data movement [57], whereas in-storage processing push computation into storage devices to ease data movement overhead [58].

Including more assemblies in the pangenome graphs increases the graph complexity in terms of traversals. The recent tools mostly prioritize data parallelism for read processing and spatial locality rather than enhancing graph operations. With hardware-friendly algorithms, cultivating concurrency by incorporating task parallelism where possible and improving resource usage through temporal and spatial locality can facilitate traversals of both complex graphs and longer reads. Further reinforcement through domain-specific hardware designs can reduce memory latency and maximize concurrent throughput. This, in turn, optimizes existing and future tools to fully harness their potential in advancing pangenome research.

Key Points

- We present an in-depth performance analysis of de Bruijn graph-based read mapping tools, evaluating their scalability, hardware utilization, and energy consumption across pangenomic datasets.
- By programmatically isolating the indexing, seeding, and alignment stages, the study evaluates how tool efficiency scales against varying thread counts and graph complexities.
- Benchmarking reveals that MetaGraph processes short reads fastest with a 100% mapping rate, whereas GraphAligner delivers superior performance and memory efficiency for long reads.
- Our findings highlight that future tools must prioritize task parallelism and data locality to overcome the high cache miss bottlenecks caused by growing pangenome graph complexity.

Acknowledgements

We thank Harun Mustafa and Nika Mansouri Ghiasi for insightful discussions on MetaGraph, and Camille Marchet and Zamin Iqbal for valuable conversations during Genome Informatics 2024.

Supplementary material

Supplementary material is available at *Briefings in Bioinformatics* online.

Conflicts of interest

C. Alkan is a co-founder of Lidya Genomics.

Funding

This work was partly supported by the European Union's Horizon Programme for Research and Innovation under grant agreement No. 101047160 (Project BioPIM).

Data availability

The datasets were derived from public domain sources, and the accessions are provided in the supplementary materials. The example scripts and code routines in all experiments are open-sourced in our GitHub repository: https://github.com/BilkentCompGen/dbg-performance-benchmarking.

References

1. Lightbody G, Haberland V, Browne F *et al.* Review of applications of high-throughput sequencing in personalized medicine: barriers and facilitators of future progress in research and clinical application. *Brief Bioinform* 2019;**20**:1795–811.
2. Sherman RM, Salzberg SL. Pan-genomics in the human genome era. *Nat Rev Genet* 2020;**21**:243–54.
3. Baaijens JA, Bonizzoni P, Boucher C *et al.* Computational graph pangenomics: a tutorial on data structures and their applications. *Nat Comput* 2022;**21**:81–108.
4. Wang S, Qian Y-Q, Zhao R-P *et al.* Graph-based pan-genomes: Increased opportunities in plant genomics. *J Exp Bot* 2023;**74**:24–39.
5. Chen N-C, Solomon B, Mun T *et al.* Reference flow: reducing reference bias using multiple population genomes. *Genome Biol* 2021;**22**:1–17.
6. Martiniano R, Garrison E, Jones ER *et al.* Removing reference bias and improving indel calling in ancient dna data analysis by mapping to a sequence variation graph. *Genome Biol* 2020;**21**:1–18.
7. Sherman RM, Forman J, Antonescu V *et al.* Assembly of a pan-genome from deep sequencing of 910 humans of African descent. *Nat Genet* 2019;**51**:30–5.
8. Computational Pan-Genomics Consortium. Computational pan-genomics: status, promises and challenges. *Brief Bioinform* 2018;**19**:118–35.
9. Andreace F, Lechat P, Dufresne Y *et al.* Comparing methods for constructing and representing human pangenome graphs. *Genome Biol* 2023;**24**:1–19.
10. Gong Y, Li Y, Liu X *et al.* A review of the pangenome: how it affects our understanding of genomic variation, selection and breeding in domestic animals? *J Anim Sci Biotechnol* 2023;**14**:73.
11. Chaisson MJP, Sanders AD, Zhao X *et al.* Multi-platform discovery of haplotype-resolved structural variation in human genomes. *Nat Commun* 2019;**10**:1784.
12. Nurk S, Koren S, Rhie A *et al.* The complete sequence of a human genome. *Science* 2022;**376**:44–53.
13. Liao W-W, Asri M, Ebler J *et al.* A draft human pangenome reference. *Nature* 2023;**617**:312–24.
14. Vollger MR, Dishuck PC, Harvey WT *et al.* Increased mutation and gene conversion within human segmental duplications. *Nature* 2023;**617**:325–34.

15. Guarracino A, Buonaiuto S, Gomes L *et al.* Recombination between heterologous human acrocentric chromosomes. *Nature* 2023;**617**:335–43.
16. Hickey G, Monlong J, Ebler J *et al.* Pangenome graph construction from genome alignments with Minigraph-Cactus. *Nat Biotechnol* 2023;**42**:1–11.
17. Porubsky D, Vollger MR, Harvey WT *et al.* Gaps and complex structurally variant loci in phased genome assemblies. *Genome Res* 2023;**33**:496–510.
18. Le DQ, Nguyen TA, Nguyen SH *et al.* Efficient inference of large prokaryotic pangenomes with PanTA. *Genome Biol* 2024;**25**:209.
19. Ruperao P, Rangan P, Shah T *et al.* Developing pangenomes for large and complex plant genomes and their representation formats. *J Adv Res* 2025;**78**:29–46.
20. Eizenga JM, Novak AM, Sibbesen JA *et al.* Pangenome graphs. *Annu Rev Genomics Hum Genet* 2020;**21**:139–62.
21. Depuydt L, Renders L, Abeel T *et al.* Pan-genome de Bruijn graph using the bidirectional FM-index. *BMC Bioinformatics* 2023;**24**:400.
22. Garrison E, Sirén J, Novak AM *et al.* Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nat Biotechnol* 2018;**36**:875–9.
23. Kaplan N, Schmelzle J-N, Yufeng G *et al.* PangenomicsBench: a benchmark suite and characterization of Pangenomics. M. Kandemir & M. Arunachalam (eds), *Proceedings of the 2025 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 272–85. Piscataway, NJ: IEEE, 2025.
24. Minkin I, Pham S, Medvedev P. TwoPaCo: an efficient algorithm to build the compacted de Bruijn graph from many complete genomes. *Bioinformatics* 2017;**33**:4024–32.
25. Holley G, Melsted P. Bifrost: highly parallel construction and indexing of colored and compacted de Bruijn graphs. *Genome Biol* 2020;**21**:1–20.
26. Almodaresi F, Zakeri M, Patro R. PuffAligner: a fast, efficient and accurate aligner based on the Pufferfish index. *Bioinformatics* 2021;**37**:4048–55.
27. Rautiainen M, Marschall T. GraphAligner: aapid and versatile sequence-to-graph alignment. *Genome Biol* 2020;**21**:253.
28. Almodaresi F, Sarkar H, Srivastava A *et al.* A space and time-efficient index for the compacted colored de Bruijn graph. *Bioinformatics* 2018;**34**:i169–77.
29. Altschul SF, Gish W, Miller W *et al.* Basic local alignment search tool. *J Mol Biol* 1990;**215**:403–10.
30. Lee C, Grasso C, Sharlow MF. Multiple sequence alignment using partial order graphs. *Bioinformatics* 2002;**18**:452–64.
31. Mustafa H, Karasikov M, Ghiasi NM *et al.* Label-guided seed-chain-extend alignment on annotated De Bruijn graphs. *Bioinformatics* 2024;**40**:i337–46.
32. Hollowell C, Caramarcu C, Strecker-Kellogg W *et al.* The effect of NUMA tunings on CPU performance. *Journal of Physics: Conference Series*, Vol. **664**, p. 092010. Bristol, UK: IOP Publishing, 2015.
33. Lameter C. NUMA (non-uniform memory access): an overview: NUMA becomes more common because memory controllers get close to execution units on microprocessors. *Queue* 2013; **11**:40–51.
34. Red Hat, Inc., Section A.12. Numactl, in *Red Hat Enterprise Linux 7 Performance Tuning Guide*, Raleigh, NC: Red Hat, Inc., 2014. Accessed on [10.12. 2025] [Online]. Available: [https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/performance_tuning_guide/sect-red_hat_enterprise_linux-performance_tuning_guide-tool_reference-numactl]
35. Perf. https://perf.wiki.kernel.org/. Accessed on [30.12.2025].
36. Linux man-pages project, *perf_event_open(2) — Linux manual page*, 2025. [Online]. Available: https://man7.org/linux/man-pages/man2/perf_event_open.2.html
37. Oliveira GF, Gómez-Luna J, Orosa L *et al.* DAMOV: a new methodology and benchmark suite for evaluating data movement bottlenecks. *IEEE Access* 2021;**9**:134457–134502.
38. Sanchez D, Kozyrakis C. ZSim: fast and accurate microarchitectural simulation of thousand-core systems. *ACM SIGARCH Computer Architecture News* 2013;**41**:475–86.
39. Kim Y, Yang W, Mutlu O. Ramulator: a fast and extensible DRAM simulator. *Conference Name: IEEE Computer Architecture Letters* **15**:45–9.
40. Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3 (3A, 3B, 3C, & 3D)*. System Programming Guide. Intel Corporation, 2022. Accessed on [10.12.2025]. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
41. Khan KN, Hirki M, Niemi T *et al.* RAPL in action: experiences in using RAPL for power measurements. *ACM Trans Model Perform Evaluation Comput Syst* 2018;**3**:1–26.
42. Weaver VM, Johnson M, Kasichayanula K *et al.* Measuring energy and power with PAPI. In: Balaji P, Lin H (eds), *2012 41st International Conference on Parallel Processing Workshops*, pp. 262–8. Piscataway, NJ: IEEE, 2012.
43. Intel, msr-tools: Utilities for reading and writing model-specific registers (MSRs), GitHub. [Online]. Available: https://github.com/intel/msr-tools
44. Karasikov M, Mustafa H, Danciu D *et al.* Efficient and accurate search in petabase-scale sequence repositories. *Nature* 2025; **647**:1036–1044.
45. GFA group. *Gfa: Graphical Fragment Assembly (Gfa) Format Specification* 2021. Available at: http://gfa-spec.github.io/GFA-spec/.
46. Coggi M, Di Donato GW, Santambrogio MD. A novel methodology for a comprehensive analysis of genomic sequence-to-graph alignment tools. *IEEE Trans Comput Biol Bioinform* 2025; **22**:228–39.
47. Limasset A, Rizk G, Chikhi R *et al.* Fast and scalable minimal perfect hashing for massive key sets. *Proceedings of the 16th International Symposium on Experimental Algorithms (SEA 2017), Leibniz International Proceedings in Informatics (LIPIcs)*, Vol.**75**, pp. 25:1–25:16, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
48. Bowe A, Onodera T, Sadakane K *et al.* Succinct de Bruijn graphs. In: Raphael B, Tang J (eds), *International Workshop on Algorithms in Bioinformatics*, pp. 225–35. Heidelberg, Germany: Springer, 2012.
49. Liu B, Guo H, Brudno M *et al.* deBGA: read alignment with de Bruijn graph-based seed and extension. *Bioinformatics* 2016;**32**:3224–32.
50. Rautiainen M, Mäkinen V, Marschall T. Bit-parallel sequence-to-graph Slignment. *Bioinformatics* 2019;**35**:3599–607.
51. Myers G. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *J ACM* 1999;**46**:395–415.
52. Navarro G. Improved approximate pattern matching on hypertext. *Theor Comput Sci* 2000;**237**:455–63.
53. Karasikov M, Mustafa H, Rätsch G *et al.* Lossless indexing with counting de Bruijn graphs. *Genome Res* 2022;**32**:1754–64.
54. Li H. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics* 2018;**34**:3094–100.
55. Kepler Project Contributors. *Kepler Energy Sources*. https://sustainable-computing.io/archive/design/kepler-energy-sources/, 2024. Accessed on [19.12. 2025].
56. Cali DS, Kanellopoulos K, Lindegger J *et al.* SeGraM: a universal hardware accelerator for genomic sequence-to-graph and

sequence-to-sequence mapping. In: Salapura V, Zahran M, Chong F (eds), *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pp. 638–55, New York, NY: ACM, 2022.
57. Simon WA, Yavits L, Koliogeorgi K *et al.* Processing-In-memory for genomics workloads. *IEEE Micro* 2026;**46**:70–80.
58. Mansouri Ghiasi N, Mustafa H, Güloglu T *et al* GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design. 2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA) 2026.
59. Garrison E, Guarracino A, Heumos S *et al.* Building pangenome graphs. *Nat Methods* 2024;**21**:2008–2012.
60. National Center for Biotechnology Information (NCBI). *BioProject PRJNA730823* 2023.
61. Paten B, Eizenga JM, Rosen YM *et al.* Superbubbles, ultrabubbles, and cacti. *J Comput Biol* 2018;**25**:649–63.
62. Li H, Feng X, Chu C. The design and construction of reference pangenome graphs with minigraph. *Genome Biol* 2020;**21**:1–19.
63. Dabbaghie F, Ebler J, Marschall T. BubbleGun: enumerating bubbles and superbubbles in genome graphs. *Bioinformatics* 2022;**38**:4217–9.
64. Shen W, Le S, Li Y *et al.* SeqKit: a cross-platform and ultrafast toolkit for FASTA/Q file manipulation. *PLoS One* 2016;**11**:e0163962.
65. Barker DJ, Natarajan RHL, Cooper MA *et al.* The IPD-IMGT/HLA database: recent developments in sequence submission. *Nucleic Acids Res* 2026;**54**:D1152–8.
66. Rhie A, Nurk S, Cechova M *et al.* The complete sequence of a human Y chromosome. *Nature* 2023;**621**:344–54.

© The Author(s) 2026. Published by Oxford University Press. This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial License (https://creativecommons.org/licenses/by-nc/4.0/), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited. For commercial re-use, please contact reprints@oup.com for reprints and translation rights for reprints. All other permissions can be obtained through our RightsLink service via the Permissions link on the article page on our site—for further information please contact journals.permissions@oup.com.
Briefings in Bioinformatics, 2026, **27**, bbag440
https://doi.org/10.1093/bib/bbag440
Case Study